

The objective of this lab is to:

1. Understand and practice pointer to object and array of objects.
2. Practice good coding conventions e.g commenting, meaningful variable and functions names, properly indented and modular code.

Instructions!

1. This is a **graded** lab, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
2. Indent your code properly.
3. Use meaningful variable and function names. **Use the camelCase notation.**
4. Save your work frequently. Make a habit of pressing **CTRL+S** after every line of code you write.
5. **Do NOT use any global or static variable(s).** However, global named constants may be used.
6. **Make sure that there are NO dangling pointers or memory leaks in your programs.**
7. **Make sure that all functions have been implemented and can be called in main function. Otherwise, no marks will be awarded for that function.**

Task 01

[40 Marks]

Create a class **MixedFraction** to store a mixed fraction. Recall that a mixed fraction has two parts integral part and fractional part, and is of following form: $3 \frac{1}{4}$

```
class MixedFraction
{
private:
    int integer;           // Integral part of the Mixed fraction.
    int numerator;         // Numerator of the fractional part.
    int denominator;       // Denominator of the fractional part. This must be non-zero.
public:
    MixedFraction ();      // Initialize data members to default values. Remember
                           // denominator must not be zero.

    MixedFraction (int a_integer, int a_nmrator, int a_dnmnator); // Initialize data members
                           // with parameter values. A mixed number can be negative or positive. If the mixed number is to be
                           // negative, only integer part can be negative, not numerator or denominator.

    MixedFraction (int a_integer); // Constructor with only integer part, set
                                   // fractional part to zero.

    MixedFraction (MixedFraction& a_mFrac); // copy constructor.

    ~ MixedFraction(); // Destructor should display message "Object is destroyed".

    void setIntegralPart(int a_intgr);
    bool setDenominator(int a_dnmnator);
    void setNumerator(int a_nmrator);
    int getIntegralPart () const;
    int getDenominator () const;
    int getNumerator ()const;

    int evaluate(); // should return the decimal equivalent of the mixed number.
    void display(); // should display the fraction in proper format i.e. 3 2/5.
    MixedFraction operator+(const MixedFraction& ); // overload binary + operator.
```

```

MixedFraction operator-(const MixedFraction& ); // overload binary - operator.
MixedFraction operator*(const MixedFraction& ); // overload binary * operator.
MixedFraction operator/(const MixedFraction& ); // overload binary / operator.
};

```

Implement the given class declaration. To overload four binary arithmetic operators, you should first convert mixed fraction to simple fraction (multiply the integer part by denominator and add that to numerator, write the resultant number as numerator) then perform required operation according to the following rules:

Addition:
$$\frac{a}{b} + \frac{c}{d} = \frac{(a * d + b * c)}{(b * d)}$$

Subtraction:
$$\frac{a}{b} - \frac{c}{d} = \frac{(a * d - b * c)}{(b * d)}$$

Multiplication:
$$\frac{a}{b} * \frac{c}{d} = \frac{(a * c)}{(b * d)}$$

Division:
$$\frac{\frac{a}{b}}{\frac{c}{d}} = \frac{(a * d)}{(b * c)}$$

In *main()* function, create 5 object of **MixedFraction** and use each function to show your work. Also, display the decimal equivalent of each mixed number.

Task 02

[25 Marks]

```
class MyString
{
private:
    char* str;          // Dynamically allocated character array to store the string.
    int length;         // Holds the length of the string (excluding the null
                        // terminator).

public:
    MyString();          // Initializes an empty string. Allocates memory for an empty string and
                        // sets length to 0.

    MyString(char* input); // Initializes the string with the given C-style string. Allocates
                        // memory and copies the input string into the object's internal storage.

    MyString(const MyString& other); // Performs deep copy of the string from another MyString
                        // object, ensuring both objects have separate memory allocations.

    ~MyString();         // Releases dynamically allocated memory and prints the message.

    MyString& operator=(const MyString& other); // Assigns one MyString object to another
                        // by copying its content. Ensures no memory leaks by first freeing existing memory.

    int getLength() const; // Returns the length of the string (excluding the null
                        // terminator).

    void display() const;  // Displays the string stored in the MyString object.

    MyString operator+(const MyString& other) const; // Concatenates two MyString objects by
                        // joining their contents and returning a new MyString object.

};
```

In the main function, initialize two MyString objects with string literals. Display the length of these string literals. Create a third MyString object by concatenating the first two MyString objects using the overloaded + operator. Display the result of the concatenation. Copy the contents of concatenated string into a new MyString object using overloaded = operator